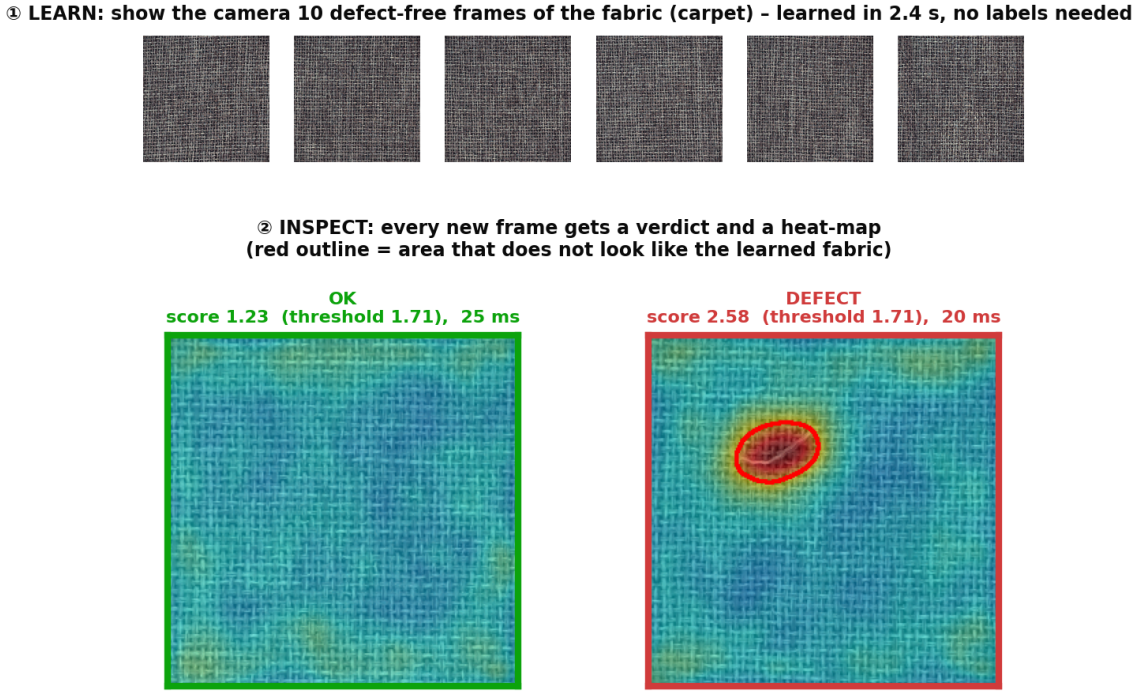
FabricInspect-IoT – Quick Guide

Fabric defect inspection that learns a new fabric from a few seconds of good material. `https://github.com/nasin0764-bit/FabricInspect-IoT`

What it does

A camera looks at the fabric just before it reaches the seaming machine. The app learns what the *good* fabric looks like from about 10 frames and then flags every frame that looks different: holes, cuts, threads, stains, contamination. It needs **no defect examples, no labelling, no GPU**, and it sets its own alarm threshold.



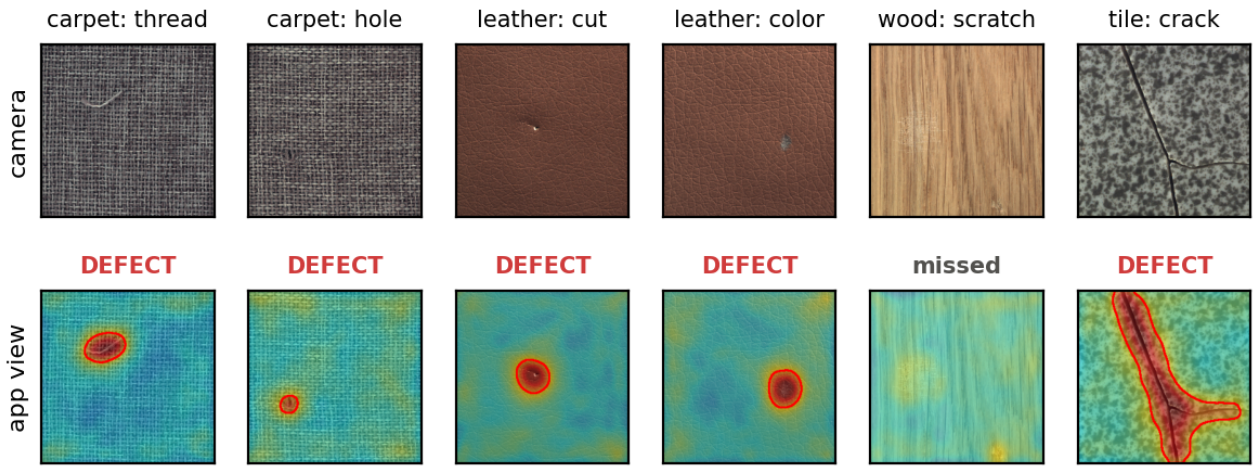
How to use it (single machine)

1. Install once: `pip install -r requirements.txt`
2. Start the app: `python -m fabricinspect gui`
3. Click **Open camera** (or **Image folder...**).
4. Type the fabric name, keep *Good shots = 10*, click **Learn fabric**, and slowly move defect-free fabric under the camera. The model and its threshold are saved as a profile.
5. Click **Start inspection**. Each frame shows **OK** or **DEFECT** with a heat-map. Defect frames are saved and every decision is written to a CSV log in `~/FabricInspect/logs`.
6. Next time the same fabric comes back, choose it under **Profile** and click **Load**.

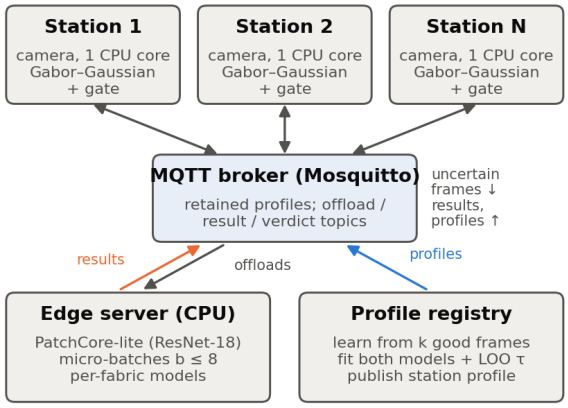
Two detectors are available: *Deep (PatchCore-lite)*, more accurate, and *Light (Gabor-Gaussian)*, a 6 kB model that runs on any small computer.

What the results look like

Real outputs of the app on public test images (10 good frames learned per material). Five of these six defects are found; the faint wood scratch is missed. The app is not perfect, and the threshold can be made stricter or looser (*margin* setting).



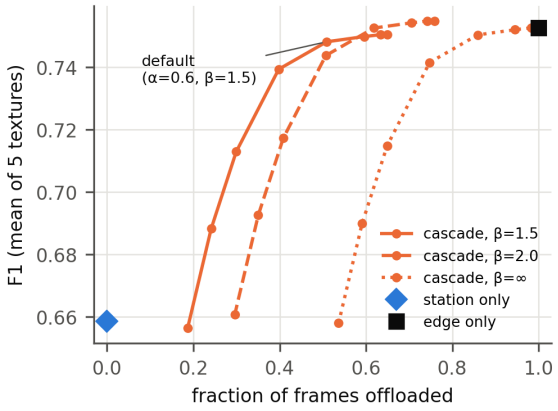
Many machines: the IoT mode



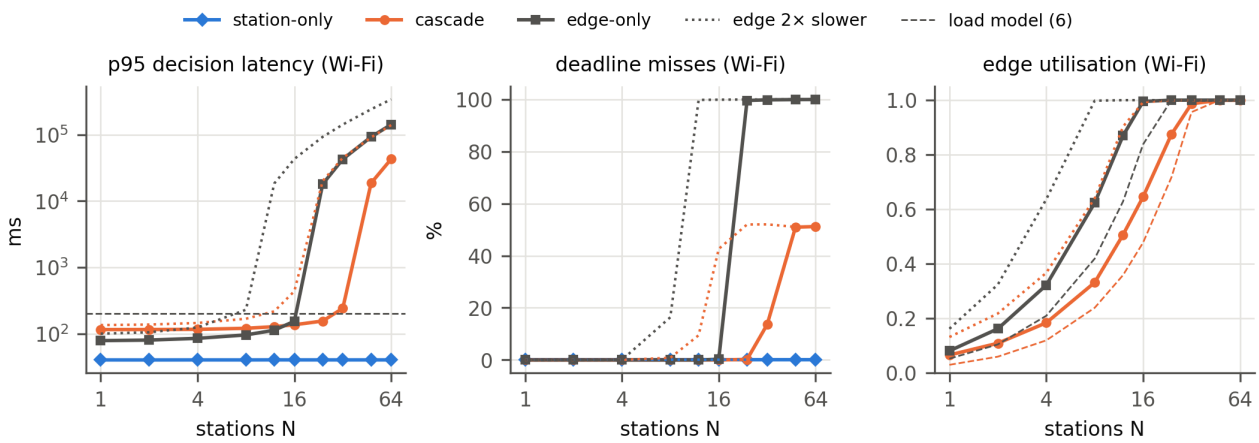
- **Stations** (one per machine) run the light model and decide clear cases themselves in about 40 ms.
- Only **uncertain** frames are sent over MQTT to one **edge server**, which checks them with the deep model.
- A fabric learned once is pushed by the **profile registry** to all machines within about 15 ms.

```
mosquitto -p 1883                                                    # message broker
python -m fabricinspect edge    --host <broker>                      # edge server
python -m fabricinspect publish --host <broker> --name denim --images good_frames/
python -m fabricinspect station --host <broker> --name st01 --fabric denim --camera 0
```

Why the gate matters



Sending every frame to the edge gives the best accuracy but loads the network and the server. The *confidence gate* keeps **99 % of that accuracy while sending only half the frames**, and sends more frames for hard fabrics (carpet: 81 %) and fewer for easy ones (leather: 23 %), automatically.



With the gate, one ordinary PC as edge server answers within the 200-ms frame time for **24 machines instead of 16**, and each machine sends 39 % less data. (Simulation built from measured timings and checked against a live test bed; see the paper.)

Good to know

- Re-learn the fabric after changing light, focus or camera; it takes seconds.
- On a slow network (about 1 Mbit/s) only the stations' own decisions arrive in time; offloaded frames are still checked, but later.
- Very fine regular meshes (like the *grid* test material) need more than 10 good frames.